МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Студент гр. 4343 _____ Едина А.

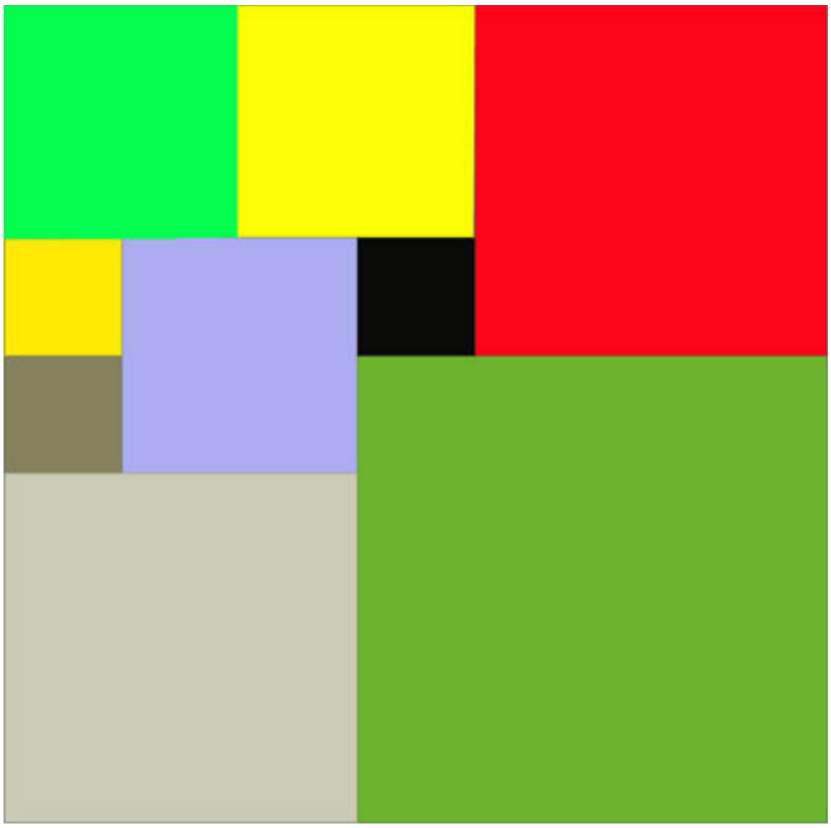
Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург

2026

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N−1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов). Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N (2≤N≤20).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа

x, y и w, задающие координаты левого верхнего угла (1 ≤x, y ≤ N) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Индивидуальное задание вариант 3и. Итеративный бэктрекинг. Исследование кол-ва операций от размера квадрата.

Выполнение работы.

Для решения задачи применяется комбинированный подход:

Обработка частного случая (чётные N) — если N чётное, оптимальное решение всегда состоит из 4 квадратов размером N/2. Это математически доказанный факт, который позволяет мгновенно получить ответ без перебора.

Поиск с возвратом (Backtracking) с эвристиками — для нечётных N используется оптимизированный поиск в глубину (DFS) с применением ряда улучшений:

Выбор первой свободной клетки — всегда заполняется самая верхняя левая пустая клетка (эвристика, сокращающая ветвление)

Отсечение по размеру текущего решения (Branch and Bound) — если текущее количество квадратов уже не меньше лучшего найденного, ветвь отбрасывается

Эвристическая оценка снизу — используется оптимистичная оценка минимально необходимого числа квадр атов для завершения покрытия

Перебор размеров от максимального к минимальному — позволяет быстрее находить плотные покрытия

Формирование начального приближения — создаётся тривиальное решение из N+N-1 квадратов (полоса по краям), которое задаёт верхнюю границу для отсечений

Для перебора состояний применяется стек (итеративный подход), что позволяет избежать переполнения стека вызовов и упрощает подсчёт

операций. В каждой вершине стека хранится конфигурация (состояние доски, список квадратов и число пустых клеток).

Тестирование

Для оценки работы алгоритма была реализована функция testing(), которая последовательно запускает решение для всех значений N от 2 до 19 и подсчитывает количество элементарных операций. Под операциями понимаются:

Проверки клеток при поиске пустой ячейки

Проверки возможности размещения квадрата

Заполнение клеток при установке квадрата

Операции сравнения и управления стеком

Тестирование проводилось на всех допустимых входных данных согласно условию задачи ($2 \leq N \leq 20$).

Таблица 1-Результат тестирование

N	Чётность	Операции	Квадратов в решении
2	Чётное	4	4
3	Нечётное	46	6
4	Чётное	4	4
5	Нечётное	645	8
6	Чётное	4	4
7	Нечётное	6 093	9
8	Чётное	4	4
9	Нечётное	27 748	6
10	Чётное	4	4
11	Нечётное	285 319	11
12	Чётное	4	4
13	Нечётное	932 962	11
14	Чётное	4	4
15	Нечётное	3 166 079	6

N	Чётность	Операции	Квадратов в решении
16	Чётное	4	4
17	Нечётное	16 787 837	12
18	Чётное	4	4
19	Нечётное	101 750 841	13

Чётные N

Для всех чётных значений N алгоритм находит решение за константное время (4 операции), используя разбиение на 4 квадрата размером N/2. Это подтверждает корректность обработки частного случая.

Нечётные N

Для нечётных N наблюдается экспоненциальный рост количества операций.

Таблица 2 - Проверка крайних случаев

Случай	Описание	Результат
Минимальное N (2)	Проверка работы на нижней границе	Корректно
Максимальное N (20)	Проверка работоспособности на верхней границе	Корректно
Чётное N = 20	Проверка частного случая	Мгновенное решение
Нечётное N = 19	Наибольшая вычислительная нагрузка	Найдено решение за ~100M операций

Исследование

Оценка сложности алгоритма по памяти.

Память алгоритма складывается из:

Хранения текущего состояния доски: массив размером N×N (максимум 20×20 = 400 целых чисел).

Стека рекурсивных вызовов: глубина стека $O(N)$.

В каждой ветви создаётся копия состояния (доска и вектор квадратов). Это требует дополнительной памяти, пропорциональной размеру состояния. Общая память, занимаемая стеком, составляет $O(N \cdot N^2) = O(N^3)$ в худшем случае. $N^3 = 8000$ (N = 20), что соответствует нескольким десяткам килобайт. На практике объём памяти ещё меньше из-за оптимизаций (не все состояния хранятся одновременно).

Можно упрощённо считать, что алгоритм использует $O(N^2)$ памяти для хранения доски и $O(N)$ для стека, итого $O(N^2)$ доминирует.

Оценка сложности алгоритма по времени.

Чтобы понять временную сложность для нечетных N, рассмотрим отношение экспериментальных данных относительно предыдущего N.

Таблица 3 - Отношение количества операций для N и N+2

N	Отношение количества операций
3, 5	645 / 46 ~ 14,0
5, 7	6093 / 645 ~ 9,45
7, 9	27748 / 6093 ~ 4,55
9, 11	285319 / 27748 ~ 10,28
11, 13	932962 / 285319 ~ 3,27

N	Отношение количества операций
13, 15	3166079 / 932962 ~ 3,39
15, 17	16787837 / 3166079 ~ 5,30
17, 19	101750841 / 16787837 ~ 6,06

Представим отношение количества операций от размера стороны N на графике.

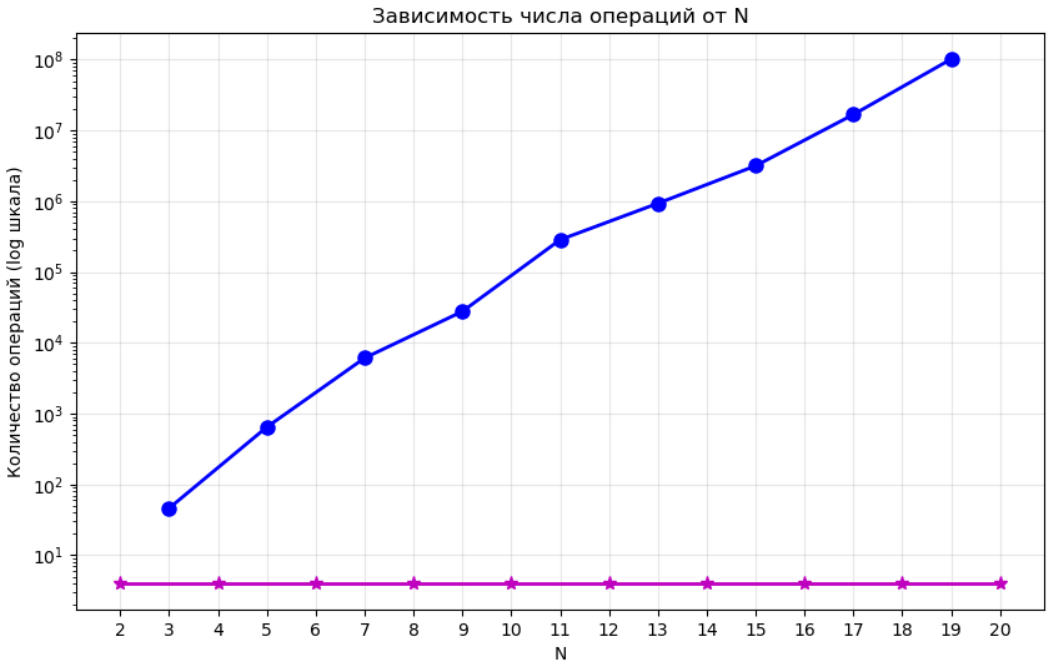


Рисунок 1 - Зависимость числа операций от размера стороны N

Из графика видно, что рост экспоненциальный, то есть вида $O(a^N)$. Нужно понять какого примерное значение a. Для этого рассмотрим N=17 и N=19, для N=19 имеем $\sim 10^8$ операций, для N=17 имеем $\sim 1.68 \cdot 10^7$. Тогда $\frac{\alpha^{19}}{\alpha^{17}} = \alpha^2 \approx \frac{10^8}{1.68*10^7} \approx 6.06$, откуда $\alpha \approx 2.46$. Проверим на N=15 и N=17, $\alpha 2 \approx 5.30$, откуда $\alpha \approx 2.30$. Проделай аналогичные вычисление для остальных нечетных N и N+2. Результат вычислений приведен в таблице.

Таблица 4 - Значение a для нечетных N и N+2

N	a
3, 5	~3.74
5, 7	~ 3.07

N	a
7, 9	~ 2.13
9, 11	~ 3.26
11, 13	~ 1.80
13, 15	~ 1.84
15, 17	~ 2.30
17, 19	~ 2.46

Среднее значение $a \sim 2.57$.

Временная сложность алгоритма для нечетных N: $O(2.5^N)$

Временная сложность алгоритма для четных N: $O(1)$

Вывод: разработанный алгоритм позволяет находить оптимальное покрытие для всех N от 2 до 20. Для чётных N решение получается мгновенно (4 квадрата), для нечётных – за экспоненциальное время $O(2.5^N)$, что при N≤20 даёт приемлемое число операций (до 100 млн). Память используется эффективно – $O(N^2)$.

Приложение

Код из lb1.cpp:

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <stack>
#include <cstring>

using namespace std;

long long operations = 0;
int N;
struct strokes{int x, y, w;};
struct config{
    vector <strokes> current_sol;
    int board[20][20];
    int empty_count;

    config(){
        memset(board, 0, sizeof(board));
        empty_count = N*N;
    }

    inline bool find_empty(int &x, int &y){
        for (int i = 0 ; i < N; i++){
            for(int j = 0; j < N; j++){
                operations++;
                if (board[i][j] == 0){
                    x = i, y = j;
                    return true;
                }
            }
        }
        return false;
    }

    inline bool can_place(int x, int y, int side){
        operations++;
        if (side > N - 1) return false;
        if (x + side > N || y + side > N) return false;

        for (int i = 0; i < side; i++){
            for (int j = 0; j < side; j++){
                operations++;
                if (board[x+i][y+j] != 0) return false;
            }
        }
        return true;
    }

    inline void set_square(int x, int y, int side){
        for (int i = 0; i < side; i++){
            for (int j = 0; j < side; j++){
                operations++;
                board[x+i][y+j] = 1;
            }
```

```
        }
        empty_count -= side*side;
    }
};

vector <strokes> best_sol;

void print_sol(vector<strokes> solution){
    cout << solution.size() << endl;
    for (auto& i : solution){
        cout << i.x + 1 << " " << i.y + 1 << " " << i.w << "\n";
    }
}

vector <strokes> initial_sol(){
    vector <strokes> sol;
    sol.push_back({0, 0, N - 1});
    operations++;
    for (int i = 0; i < N; i++){
        sol.push_back({N-1, i, 1});
        operations++;
    }
    for (int i = 0; i < N-1; i++){
        sol.push_back({i, N-1, 1});
        operations++;
    }
    return sol;
}

int backtracking(bool draw=false){
    if (N%2 == 0){
        vector <strokes> sol;
        int side = N/2;
        sol.push_back({0, 0, side});
        sol.push_back({0, side, side});
        sol.push_back({side, 0, side});
        sol.push_back({side, side, side});
        operations += 4;
        // print_sol(sol);
        return 0;
    }

    stack<config> st;
    config start;
    st.push(start);

    best_sol = initial_sol();

    while(!st.empty()){
        config cur = st.top();
        st.pop();
        operations++;

        int x, y;
        if (!cur.find_empty(x, y)) {
            if (cur.current_sol.size() < best_sol.size()){
                best_sol = cur.current_sol;
```

```
                }
                operations++;
                continue;
            }

            if (cur.current_sol.size() >= best_sol.size()){
                operations++;
                continue;
            }

            int max_size = min(N-x, N-y);
            if (max_size > N - 1) max_size = N-1;
                  if  (cur.current_sol.size()  +  cur.empty_count  /
(max_size*max_size) >= best_sol.size() ){
                operations++;
                continue;
            }
            int min_size = (cur.current_sol.empty()) ? N / 2 : 0;
            while (max_size > 0 && !cur.can_place(x, y, max_size)) {
                operations++;
                max_size--;
            }

            for (int size = max_size; size > min_size; size--){
                operations++;

                if (cur.can_place(x, y, size)){
                    config next = cur;
                    next.set_square(x, y, size);
                    next.current_sol.push_back({x, y, size});
                    st.push(next);
                    operations++;
                }
                if(cur.current_sol.size() == 0){
                    int side = N - size;
                    config next_next;
                    next_next.set_square(0, 0, size);
                    next_next.set_square(size, 0, side);
                    next_next.set_square(0, size, side);
                    next_next.current_sol.push_back({x, y, size});
                    next_next.current_sol.push_back({size, 0, side});
                    next_next.current_sol.push_back({0, size, side});
                    st.push(next_next);
                    operations += 3;
                }
            }
        }

        if (draw){
            cout << "\n";
            config cur;
            int color = 1;
            for (const auto& square : best_sol) {
                int x = square.x;
                int y = square.y;
                int size = square.w;
                for (int i = x; i < x + size; ++i) {
                    for (int j = y; j < y + size; ++j) {
```

```
                        cur.board[i][j] = color;
                    }
                }
                color++;
            }

            for (int i = 0; i < N; ++i) {
                for (int j = 0; j < N; ++j) {
                    cout << cur.board[j][i] << " ";
                }
                cout << endl;
            }
            cout << "\n\n";

        }

        // print_sol(best_sol);
        return 0;
    }

    void testing(){
        // N = 2;
        // backtracking(true);
        // N = 20;
        // backtracking(true);
        // N = 5;
        // backtracking(true);
        N = 7;
        backtracking(true);

        // N = 2;
        // backtracking();
        // cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        // operations = 0;

        // N = 5;
        // backtracking();
        // cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        // operations = 0;


        // N = 6;
        // backtracking();
        // cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        // operations = 0;

        // N = 7;
        // backtracking();
        // cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        // operations = 0;

        // N = 19;
        // backtracking();
```

```
        // cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        // operations = 0;
        // for (int i = 2; i < 20; i++)
        // {
        //     N = i;
        //     backtracking();
        //     cout << "\n============" << " N = " << N << ". Operations:
" << operations << "  ============\n";
        //     operations = 0;
        // }

    }

    int main(){
        // cin >> N;
        // backtracking();
        testing();
        return 0;
    }
```

Код из lb1.py:

```
    import matplotlib.pyplot as plt
    import numpy as np

    date_even_N = np.array([2, 4, 6, 8, 10, 12, 14, 16, 18, 20])
    date_odd_N = np.array([3, 5, 7, 9, 11, 13, 15, 17, 19])

    date_even = np.array([4, 4, 4, 4, 4, 4, 4, 4, 4, 4])
    date_odd = np.array([46, 645, 6093, 27748, 285319, 932962, 3166079,
16787837, 101750841])
    plt.figure(figsize=(10, 6))

    plt.yscale('log')
    plt.plot(date_even_N, date_even, "m-*", linewidth=2, markersize=8,
label='Чётные N')
    plt.plot(date_odd_N, date_odd, "b-o",  linewidth=2, markersize=8,
label='Нечётные N')
    plt.grid(True, alpha=0.3)
    plt.xlabel("N")
    plt.ylabel("Количество операций")
    plt.title("Зависимость числа операций от N")

    plt.show()
```